

Our test suite had 703 element getters.

55

page classes

703

element getters

2

functions with a
data contract

How you operate a dropdown –
written down up to 55 times.

A field report

But volume was not the problem.

The specification
lived in TypeScript.

```
// somewhere inside a .spec.ts  
const CASES = [ ... ]
```

And that belongs to whoever writes TypeScript –
not to whoever knows the domain.

So we moved *it* out.

The decision table
is the source.
One runner reads the
suites as data.

No generated spec files left over to maintain.
One entry point per app – not one per screen.

A precondition is an axis, not a path.

AS SCENARIOS

AS AN AXIS

		normal	proposal
create booking → book over it			
	free	✓	✓
create absence → assign anyway			
	booked	conflict	no conflict ★
expire cert → assign anyway			

★ Nobody writes that case as a script.
It only occurs to you when you see the grid.

This test was green for two days.

```
// the field had been renamed to `num`  
const seen = rows.map(r => r.number)  
// → [undefined, undefined, ...]  
  
expect(seen).toEqual([...seen].sort())  
✓ passes
```

A dead reader doesn't make an assertion *fail*.
It makes it **empty** – and empty looks exactly like correct.

Behind it: sorting ascending returned descending.

Where we stand.

	BEFORE	NOW
hand-written spec files	16	0
decision tables	—	117
test cases	—	1,981

The full report — including the two assertions **we got wrong ourselves** and what tables don't solve:

nanook.xhub.io/blog